代码总结

重邮802数据结构 · 主讲笔记

数据结构 · 考研笔记

目 录

全局约定（阅读前必看）

线性表

- 顺序表
- 链表

栈和队列

- 顺序栈
- 循环队列

串

- 存储结构
- 基本操作

树与二叉树

- 存储结构
- 基本操作
- 经典算法
- 线索二叉树

图

- 存储结构
- 经典算法

查找

- 折半查找

排序

- 插入排序
- 交换排序
- 选择排序

真题汇总

- 2026
- 2025
- 2024
- 2023
- 2022
- 2021
- 2020

全局约定（阅读前必看）

802考的是伪代码，重思路轻语法，但**约定必须自己前后统一**，否则改一行就全错。本文统一如下：

1. **下标基准分两套，按章节区分**：
   - 线性表/查找章节：数据从 `data[0]` 开始存，第 i 个元素是 `data[i-1]`；
   - 排序章节：`data[0]` 留作哨兵/暂存位，数据从 `data[1]` 存到 `data[length]`（这是教材惯例，堆排序要求 `2i` 是左孩子，必须从1开始）。 每道题开头先写一句"本题下标从0/1起"，阅卷不会挑刺，但自相矛盾会扣分。
2. **链表默认带头结点**，`L` 指向头结点，`L->next` 是第一个数据结点；不带头结点的变体单独标注。
3. **顺序栈 `top` 初值为 -1**（`S.data[++S.top]` 入栈）；**循环队列牺牲一个单元**判满（`(rear+1)%MaxSize==front`）。
4. **栈/队列作为工具使用时，元素类型按需替换**：本文 `SqStack` / `SqQueue` 的 `data` 写成 `int`，但在树、图算法里存放的是 `BiTree`、`LNode *`、顶点编号等。考场上直接写 `SqStack S; Push(S, p);` 即可，无需重新定义，**但要在答题时说明一句"栈中存放结点指针"**。
5. **`int a[n] = {0}` 是伪代码简写**：C语言中变长数组不允许带初始化列表，严格写法是先声明再用循环赋初值。802允许这样简写，但**赋非0初值（如 `{INF}`）时必须写循环**——`{INF}` 只会初始化第一个元素，其余为0，这是真实的逻辑错误。
6. 参数用 C++ 引用 `&` 表示"传入传出"，等价于 C 的二级指针，伪代码中允许。

线性表

顺序表

类型定义

- 静态顺序表

```
#define MaxSize 1000
typedef struct{
    int data[MaxSize];
    int length;
}SqList;
```

- 动态顺序表

```
typedef struct{
    int *data;
    int MaxSize, length;
}SeqList;
```

基本操作

- 初始化

```
bool InitList(SqList &L){
    for(int i = 0;i < MaxSize;i++) L.data[i] = 0;
    L.length = 0;                       // 结构体类型用.访问属性
    return true;
}
bool InitList(SeqList &L, int InitSize){
    L.data = (int *)malloc(sizeof(int) * InitSize);
    if(L.data == NULL) return false;
    L.length = 0;
    L.MaxSize = InitSize;
    return true;
}
```

- 查找

```
int LocateElem(SqList &L, int key){
    for(int i = 0;i < L.length;i++){
        if(L.data[i] == key) return i + 1;
    }
    return -1;
}
```

- 按位插入和按位删除

```
bool ListInsert(SqList &L, int i, int elem){
    if(i < 1 || i > L.length + 1) return false;
    if(L.length >= MaxSize) return false;
    for(int j = L.length;j >= i;j--) L.data[j] = L.data[j - 1];
    L.data[i - 1] = elem;
    L.length++;
    return true;
}
bool ListDelete(SqList &L, int i, int &elem){
    if(i < 1 || i > L.length) return false;
    elem = L.data[i - 1];
    for(int j = i;j < L.length;j++) L.data[j - 1] = L.data[j];
    L.length--;
    return true;
}
```

链表

类型定义

- 单链表

```
typedef struct LNode{
    int data;
    struct LNode *next;
}LNode, *LinkList;
```

- 双链表

```
typedef struct DNode{
    int data;
    struct DNode *prior, *next;
}DNode, *DLinkList;
```

带头结点的基本操作

- 初始化

```
bool InitList(LinkList &L){
    L = (LNode *)malloc(sizeof(LNode));
    if(L == NULL) return false;
    L->next = NULL;                              // 结构体指针用->访问属性
    return true;
}
```

- 查找

```
LNode *GetElem(LinkList L, int i){
    if(i < 0) return NULL;
    LNode *p = L;
    int j = 0;
    while(j < i && p != NULL){
```

```
        p = p->next;
        j++;
    }
    return p;
}
LNode *LocateElem(LinkList L, int elem){
    LNode *p = L->next;
    while(p != NULL && p->data != elem) p = p->next;
    return p;
}
// 双链表的按位查找：结构不同，必须单独写一个，不能直接复用上面的LNode版本
DNode *GetElem(DLinkList L, int i){
    if(i < 0) return NULL;
    DNode *p = L;
    int j = 0;
    while(j < i && p != NULL){
        p = p->next;
        j++;
    }
    return p;
}
```

- 按位插入

```
bool ListInsert(LinkList &L, int i, int elem){
    LNode *p = GetElem(L, i - 1);
    if(p == NULL) return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if(s == NULL) return false;
    s->data = elem;
    s->next = p->next;
    p->next = s;
    return true;
}
bool ListInsert(DLinkList &L, int i, int elem){
    DNode *p = GetElem(L, i - 1);
    if(p == NULL) return false;
    DNode *s = (DNode *)malloc(sizeof(DNode));
    if(s == NULL) return false;
    s->data = elem;
    s->next = p->next;
    s->prior = p;
    if(p->next != NULL) p->next->prior = s;
    p->next = s;
    return true;
}
```

- 按位删除

```
bool ListDelete(LinkList &L, int i, int &elem){
    LNode *p = GetElem(L, i - 1);
    if(p == NULL) return false;
    if(p->next == NULL) return false;
    LNode *s = p->next;
    elem = s->data;
```

```
    p->next = s->next;
    free(s);
    return true;
}
bool ListDelete(DLinkList &L, int i, int &elem){
    DNode *p = GetElem(L, i - 1);
    if(p == NULL) return false;
    if(p->next == NULL) return false;
    DNode *s = p->next;
    elem = s->data;
    p->next = s->next;
    if(s->next != NULL) s->next->prior = p;
    free(s);
    return true;
}
```

经典算法

交换相邻的两个结点

```
// 前提：q紧跟在p之后，且p有前驱、q有后继（带头结点或循环双链表可保证）
// 六条赋值的次序不能乱：先用旧指针把外部接好，最后再改p、q自身的互指
void exchange(DNode *p, DNode *q){
    p->prior->next = q;
    q->next->prior = p;
    p->next = q->next;
    q->prior = p->prior;
    p->prior = q;
    q->next = p;
}
```

原地逆置链表

```
void InvertedList(LinkList &L){
    if(L == NULL || L->next == NULL) return;
    LNode *p = L->next, *temp;
    L->next = NULL;
    while(p != NULL){
        temp = p;
        p = p->next;
        temp->next = L->next;
        L->next = temp;
    }
}
// 循环单链表的原地逆置
void InvertedList(LinkList &L){
    if(L == NULL || L->next == NULL) return;
    LNode *p = L->next, *temp;LNode *rear = p;
    L->next = NULL;
    while(p != L){
        temp = p;
        p = p->next;
        temp->next = L->next;
        L->next = temp;
```

```
    }
    rear->next = L;
}
```

判断回文串

```
bool IsPalindrome(LinkList L, int n){
    SqStack s;InitStack(s);
    LNode *q, *p = L->next;
    for(int i = 1;i <= n / 2;i++){          // 将前半部分放入栈中
        Push(s, p);
        p = p->next;
    }
    if(n % 2 == 1) p = p->next;             // 奇数跳过中点元素
    while(p != NULL){
        Pop(s, q);
        if(p->data != q->data) break;
        else p = p->next;
    }
    return IsEmpty(s);                      // 栈空则为回文串
}
```

输出链表倒数第k个元素

```
// 方法一：快慢针
int findK(LinkList &L, int k){
    LNode *p = L, *q = L;
    while(k != 0 && q != NULL){
        q = q->next;
        k--;
    }
    if(q == NULL) return 0;
    while(q != NULL){
        q = q->next;
        p = p->next;
    }
    printf("%d", p->data);
    return 1;
}
// 方法二：运用栈
int findK(LinkList &L, int k){
    SqStack S;InitStack(S);
    LNode *p = L->next;
    while(p != NULL){
        Push(S, p);
        p = p->next;
    }
    int i = 0;
    while(!IsEmpty(S) && i < k){
        Pop(S, p);
        i++;
    }
    if(i == k){
        printf("%d", p->data);
```

```
        return 1;
    }
    else return 0;
}
```

归并两个递增表为递减单链表

```
bool Union(LinkList &la, LinkList lb){
    if(la == NULL || lb == NULL) return false;      // 任一为空就不能解引用，必须用||
    LNode *pa = la->next, *pb = lb->next;
    la->next = NULL;                                          // 以la作为新链表
    while(pa != NULL && pb != NULL){
        LNode *temp;
        if(pa->data <= pb->data){
            temp = pa;
            pa = pa->next;
        }else{
            temp = pb;
            pb = pb->next;
        }
        temp->next = la->next;
        la->next = temp;
    }
    while(pa != NULL){
        LNode *temp = pa;
        pa = pa->next;
        temp->next = la->next;
        la->next = temp;
    }
    while(pb != NULL){
        LNode *temp = pb;
        pb = pb->next;
        temp->next = la->next;
        la->next = temp;
    }
    free(lb);                                  // lb的头结点已无用，顺手释放
    return true;
}
```

> 思路要点：**归并成递减表，用头插即可自然反序**，不需要先归并再逆置。这也是"递增表归并成递减表"这一类题的通用套路。

最快合并两个循环单链表

```
/* 核心思想：将较短表的尾指向较长表的第一个元素，再将较长表的头指向较短表的第一个元素，最后释放较短表头结点 */
bool Union(LinkList L1, LinkList L2, int m, int n){
    if(m <= 0 || n <= 0) return false;
    if(m < n){
        LNode *p = L1->next;
        while(p->next != L1) p = p->next;
        p->next = L2->next;
        L2->next = L1->next;
```

```
        free(L1);
    }else{
        LNode *p = L2->next;
        while(p->next != L2) p = p->next;
        p->next = L1->next;
        L1->next = L2->next;
        free(L2);
    }
    return true;
}
```

408真题19年41题

设线性表L=(a1,a2,a3,...,an)采用带头结点的单链表存储，现在需要设计一个空间复杂度为O(1)且时间上尽可能高效的算法，重新排列L中的各结点，得到线性表L'=(a1,an,a2,an-1,...)

```
typedef struct node{
    int data;
    struct node *next;
}NODE;
void Reorder(NODE *head){
    if(head == NULL || head->next == NULL || head->next->next == NULL) return;
    NODE *slow = head->next, *fast = head->next;
    while(fast != NULL && fast->next != NULL){
        slow = slow->next;
        fast = fast->next->next;
    }

    NODE *second = slow->next, *p = second->next;
    slow->next = NULL;
    while(p != NULL){
        NODE *temp = p->next;
        p->next = second->next;
        second->next = p;
        p = temp;
    }

    NODE *first = head->next;
    while(second != NULL){
        NODE *temp1 = first->next, *temp2 = second->next;
        second->next = temp1;first->next = second;
        first = temp1;second = temp2;
    }
}
```

栈和队列

栈和队列是较为经典的算法工具，在很多场景下都需要借助这二者。作为工具使用时，一般只需进行简单的初始化和基础操作，整体考察难度不大，但要注意一些定义细节，例如栈顶指针的初值是 top=0 还是 top=-1。

顺序栈

类型定义

```
typedef struct{
    int data[MaxSize];
    int top;
}SqStack;
```

基本操作

- 初始化

```
bool InitStack(SqStack &S){
    S.top = -1;
    return true;
}
```

- 判空和判满

```
bool IsEmpty(SqStack &S){
    return S.top == -1;
}
bool IsFull(SqStack &S){
    return S.top == MaxSize - 1;
}
```

- 求长度

```
int GetLength(SqStack &S){
    return S.top + 1;
}
```

- Push，Pop和GetTop

```
bool Push(SqStack &S, int elem){
    if(S.top == MaxSize - 1) return false;
    S.data[++S.top] = elem;
    return true;
}
bool Pop(SqStack &S, int &elem){
    if(S.top == -1) return false;
    elem = S.data[S.top--];
```

```
    return true;
}
bool GetTop(SqStack &S, int &elem){
    if(S.top == -1) return false;
    elem = S.data[S.top];
    return true;
}
```

循环队列

类型定义

```
typedef struct{
    int data[MaxSize];
    int rear, front;
}SqQueue;
```

基本操作

- 初始化

```
bool InitQueue(SqQueue &Q){
    Q.front = 0;
    Q.rear = 0;
    return true;
}
```

- 判空、判满和求长度

```
bool IsEmpty(SqQueue &Q){
    return Q.rear == Q.front;
}
bool IsFull(SqQueue &Q){
    return (Q.rear + 1) % MaxSize == Q.front;
}
int GetLength(SqQueue &Q){
    return (Q.rear - Q.front + MaxSize) % MaxSize;
}
```

- EnQueue，DeQueue和GetFront

```
bool EnQueue(SqQueue &Q, int elem){
    if((Q.rear + 1) % MaxSize == Q.front) return false;
    Q.data[Q.rear] = elem;
    Q.rear = (Q.rear + 1) % MaxSize;
    return true;
}
bool DeQueue(SqQueue &Q, int &elem){
    if(Q.rear == Q.front) return false;
    elem = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;
    return true;
```

```
}
int GetFront(SqQueue &Q){
    return Q.data[Q.front];
}
```

串

存储结构

- 定长顺序存储（考察串的基本概念时用这种，不涉及malloc）

```
typedef struct{
    char ch[MaxSize];       // 定长数组，空间在定义时就固定
    int length;
}SString;
```

- 堆分配存储（**下面的基本操作全部基于这一种**，因为要用malloc动态申请空间）

```
typedef struct{
    char *ch;               // 指针，指向动态申请的字符数组
    int length;
}String;
```

> 【易错】两种定义不能混用：定长数组 `char ch[MaxSize]` 不能 `free`，也不能被赋值为 `NULL`。凡是出现 `str.ch = (char*)malloc(...)` 的操作，其结构体的 ch 必须是**指针**。

- 链式串

```
typedef struct LNode{
    char data;
    struct LNode *next;
}LNode, *LinkList;
```

基本操作

- 赋值

```
int strassign(String &str, char *ch) {
    if(str.ch) free(str.ch);                        // 清空
    int len = 0;
    char *c = ch;
    while(*c){ len++; c++; }
    if(len == 0){                                   // 空串则赋值NULL和0
        str.ch = NULL;
        str.length = 0;
        return 0;
    }else{
        str.ch = (char*)malloc(sizeof(char) * (len + 1));
        if(str.ch == NULL) return 0;
        c = ch;
        for(int i = 0;i <= len; i++){           // <=因为要把结束符'\0'存入
            str.ch[i] = *c;
            c++;
        }
```

```
        str.length = len;
        return 0;
    }
}
```

- 比较

```
bool strcompare(String s1, String s2){
    for(int i = 0;i < s1.length && i < s2.length; i++){
        if(s1.ch[i] != s2.ch[i]) return false;
    }
    return (s1.length - s2.length) == 0;                    // 退出遍历时两串要长度一致
}
```

- 拼接

```
int concat(String &str, String str1, String str2){
    if(str.ch){ free(str.ch); str.ch = NULL; }
    str.ch = (char*)malloc(sizeof(char) * (str1.length + str2.length + 1));
    if(str.ch == NULL) return 0;

    int i = 0;
    while(i < str1.length){                                 // 这里不需要存入结束符'\0'
        str.ch[i] = str1.ch[i];
        i++;
    }
    int j = 0;
    while(j <= str2.length){                                // 这里要存入结束符'\0'
        str.ch[i + j] = str2.ch[j];
        j++;
    }
    str.length = str1.length + str2.length;
    return 0;
}
```

- 求子串

```
int substring(String *substr, String str, int pos, int len){
    if(pos < 0 || pos >= str.length || len < 0 || pos + len > str.length) return 1;
    if(substr->ch){
        free(substr->ch);
        substr->ch = NULL;
    }
    if(len == 0){
        substr->ch = NULL;
        substr->length = 0;
        return 0;
    }else{
        substr->ch = (char*)malloc(sizeof(char) * (len + 1));
        int i = pos;
        int j = 0;
        while(i < pos + len){
            substr->ch[j] = str.ch[i];
            i++;
            j++;
```

```
        }
        substr->ch[j] = '\0';
        substr->length = len;
        return 0;
    }
}
```

- BF朴素匹配

```
int index(String str, String substr){
    if(str.length == 0 || substr.length == 0) return 0;
    int i = 0, j = 0, k = i;
    while(i < str.length && j < substr.length){
        if(str.ch[i] == substr.ch[j]) { i++; j++; }
        else{ j = 0; i = ++k; }
    }
    if(j == substr.length) return k + 1;
    return 0;
}
```

树与二叉树

存储结构

- 二叉链表

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild, *rchild;
}BiTNode, *BiTree;
```

- 孩子兄弟链表

```
typedef struct CSNode {
    ElemType data;
    struct CSNode *firstchild;   // 第一个孩子
    struct CSNode *nextbro;  // 下一个兄弟
} CSNode, *CSTree;
```

基本操作

- 递归先序遍历

```
void PreOrderTraverse(BiTree T){
    if(T == NULL) return;
    visit(T);
    PreOrderTraverse(T->lchild);
    PreOrderTraverse(T->rchild);
}
```

- 非递归先序遍历

```
void PreOrderTraverse(BiTree T){
    Stack s;InitStack(s);
    if(T == NULL) return;
    Push(s, T);
    BiTree p;
    while(!IsEmpty(s)){
        Pop(s, p);visit(p);
        if(p->rchild != NULL) Push(s, p->rchild);
        if(p->lchild != NULL) Push(s, p->lchild);
    }
}
```

- 非递归中序遍历

```
void InOrderTraverse(BiTree T){
    if(T == NULL) return;
    Stack s;InitStack(s);
    BiTNode *p = T;
```

```
    while(!IsEmpty(s) || p != NULL){
        while(p != NULL){
            Push(s, p);
            p = p->lchild;
        }
        if(!IsEmpty(s)){
            Pop(s, p);
            visit(p);
            p = p->rchild;
        }
    }
}
```

- 非递归后序遍历

```
void PostOrderTraverse(BiTree T){
    Stack s1, s2;InitStack(s1);InitStack(s2);
    BiTNode *p;
    Push(s1, T);
    // 用前序的方式遍历，将遍历序列按序存入栈中逆置读取，得到后序遍历序列
    while(!IsEmpty(s1)){
        Pop(s1, p);Push(s2, p);
        if(p->lchild) Push(s1, p->lchild);
        if(p->rchild) Push(s1, p->rchild);
    }
    while(!IsEmpty(s2)){
        Pop(s2, p);
        visit(p);
    }
}
```

- 层序遍历

```
void LayeredOrderTraverse(BiTree T){
    if(T == NULL) return;
    SqQueue Q;InitQueue(Q);
    EnQueue(Q, T);
    BiTNode *p;
    while(!IsEmpty(Q)){
        DeQueue(Q, p);visit(p);
        if(p->lchild != NULL) EnQueue(Q, p->lchild);
        if(p->rchild != NULL) EnQueue(Q, p->rchild);
    }
}
```

- 求树的度

```
int degree = 0;                     // 全局变量
int Post(CSNode *p){
    if(p == NULL) return 0;
    Post(p->firstchild);
    int d = 1 + Post(p->nextbro);
    degree = max(degree, d);
```

```
    return d;
}
```

- 求树的高度

```
int TreeDepth(BiTree T){
    if (T == NULL) return 0;
    return max(TreeDepth(T->lchild), TreeDepth(T->rchild)) + 1;
}
```

- 求树的宽度

```
int Width(BiTree T){
    if(T == NULL) return 0;
    int front = 0, rear = -1, last = 0;      // last记录同层最右结点的位置
    BiTree Q[MaxSize];BiTree p;
    Q[++rear] = T;
    int temp = 0, maxwidth = 0;
    while(front <= last){                    // 保证循环在同层内
        p = Q[front++];temp++;
        if(p->lchild != NULL) Q[++rear] = p->lchild;
        if(p->rchild != NULL) Q[++rear] = p->rchild;
        if(front > last){
            last = rear;                         // 当一层遍历结束，当前的队尾元素就是下一层的最右结
点
            if(temp > maxwidth) maxwidth = temp;
            temp = 0;
        }
    }
    return maxwidth;
}
```

- 统计结点总数

```
int CountNodes(BiTree T){
    if(T == NULL) return 0;
    else return CountNodes(T->lchild) + CountNodes(T->rchild) + 1;
}
```

- 统计叶结点数

```
int CountLeaves(BiTree T){
    if(T == NULL) return 0;
    if(T->lchild == NULL && T->rchild == NULL) return 1;
    else return CountLeaves(T->lchild) + CountLeaves(T->rchild);
}
// 孩子兄弟表示法
int CountLeaves(CSTree T){
    if(T == NULL) return 0;
    if(T->firstchild == NULL) return 1 + CountLeaves(T->nextbro);
    return CountLeaves(T->firstchild) + CountLeaves(T->nextbro);
}
```

经典算法

判断完全二叉树

```
// 在完全二叉树中，若某个结点缺少子结点，则其后所有结点都必须是叶子结点
int JudgeComplete(BiTree T){
    if(T == NULL) return 1;
    SqQueue Q;
    InitQueue(Q);
    EnQueue(Q, T);
    int tag = 0;BiTree p;
    while(!IsEmpty(Q)){
        DeQueue(Q, p);
        if(p->lchild != NULL){
            if(tag) return 0;
            EnQueue(Q, p->lchild);
        }else{ tag = 1; }
        if(p->rchild != NULL){
            if(tag) return 0;
            EnQueue(Q, p->rchild);
        }else{ tag = 1; }
        }
        return 1;
}
```

判断平衡二叉树

```
int JudgeAVL(BiTree T){
    if(T == NULL) return 0;

    int HL = JudgeAVL(T->lchild);
    if(HL == -1) return -1;
    int RL = JudgeAVL(T->rchild);
    if(RL == -1) return -1;

    if(HL - RL < -1 || HL - RL > 1) return -1;
    else return max(HL, RL) + 1;
}
```

求镜像树

```
void mirrorTree(BiTree &T){
    if(T == NULL) return;
    BiTNode *temp = T->rchild;
    T->rchild = T->lchild;
    T->lchild = temp;
    mirrorTree(T->lchild);
    mirrorTree(T->rchild);
}
```

线索二叉树

结构定义

```
typedef struct BiThrNode{
    int data;
    struct BiThrNode *lchild, *rchild;
    int ltag, rtag;
}BiThrNode, *BiThrTree;
```

二叉树的中序线索化

```
void InThread(BiThrTree &p, BiThrTree &pre){
    if(p != NULL){
        InThread(p->lchild, pre);

        if(p->lchild == NULL){
            p->lchild = pre;
            p->ltag = 1;
        }
        if(pre != NULL && pre->rchild == NULL){
            pre->rchild = p;
            pre->rtag = 1;
        }
        pre = p;

        InThread(p->rchild, pre);
    }
}
void CreateInThread(BiThrTree &T){
    BiThrNode *pre = NULL;
    if(T){
        InThread(T, pre);
        if(pre != NULL){
            pre->rchild = NULL;
            pre->rtag = 1;
        }
    }
}
```

图

存储结构

- 邻接矩阵

```
#define MaxVertexNum 100
typedef struct{
    char vex[MaxVertexNum];
    int edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
    int kind;                   // 表示有向或无向
}MGraph;
```

- 邻接表

```
typedef struct ArcNode{         // 边表结点
    int adjvex;
    // int weight;若边有权值
    struct ArcNode *nextarc;
}ArcNode;
typedef struct VNode{           // 顶点结点
    VertexType data;
    ArcNode *firstarc;
}VNode, AdjList[MaxVertexNum];
typedef struct{
    AdjList vertices;
    int vexnum, arcnum;
    int kind;                   // 1表有向
}ALGraph;
```

经典算法

求所有结点的入度

```
void Indegree(ALGraph G, int indeg[]){
    for(int i = 0; i < G.vexnum; i++) indeg[i] = 0;
    for(int i = 0;i < G.vexnum;i++){
        for(ArcNode *p = G.vertices[i].firstarc;p != NULL;p = p->nextarc) indeg[p->adjvex]++;
    }
}
```

邻接表转邻接矩阵

```
int vexs[G.vexnum][G.vexnum] = {0};
for(int i = 0;i < G.vexnum;i++){
    for(ArcNode *p = G.vertices[i].firstarc;p != NULL;p = p->nextarc){
        vexs[i][p->adjvex] = 1;vexs[p->adjvex][i] = 1;
    }
```

```
/*
或使用以下遍历方式
ArcNode *p = G->vertices[i].firstarc;
    while (p) {
        int v = p->adjvex;
        A[i][v] = 1;
        A[v][i] = 1;
        p = p->nextarc;
    }
*/
}
```

深度优先搜索

```
bool visited[MAX_VERTEX_NUM];
void DFS(ALGraph G, int v){
    visit(v);
    visited[v] = true;
    for(ArcNode *p = G.vertices[v].firstarc;p != NULL;p = p->nextarc){
        if(!visited[p->adjvex]) DFS(G, p->adjvex);
    }
}
void DFS_Traverse(ALGraph G){
    for(int i = 0;i < G.vexnum;i++) visited[i] = false;
    for(int i = 0;i <G.vexnum;i++){
        if(!visited[i]) DFS(G, i);
    }
}
```

广度优先搜索

```
bool visited[MAX_VERTEX_NUM];
SqQueue Q;
void BFS(ALGraph G, int v){
    visit(v);visited[v] = true;EnQueue(Q, v);
    while(!IsEmpty(Q)){
        DeQueue(Q, v);
        for(ArcNode *p = G.vertices[v].firstarc;p != NULL;p = p->nextarc){
            int k = p->adjvex;
            if(!visited[k]){
                visit(k);visited[k] = true;EnQueue(Q, k);
            }
        }
    }
}
void BFS_Traverse(ALGraph G){
    for(int i = 0;i < G.vexnum;i++) visited[i] = false;
    InitQueue(Q);
    for(int i = 0;i <G.vexnum;i++){
        if(!visited[i]) BFS(G, i);
    }
}
```

Prim算法

```
const int INF = 0x3f3f3f3f;
void Prim(ALGraph &G) {
    bool include[G.vexnum];
    int dist[G.vexnum], parent[G.vexnum];                          // parent记录生成树中该点的父结点
    // 注意：不能写 int dist[] = {INF}，那样只有第一个元素是INF，其余为0
    for(int i = 0;i < G.vexnum;i++){
        include[i] = false;
        dist[i] = INF;
        parent[i] = -1;
    }

    dist[0] = 0;
    for (int k = 0; k < G.vexnum; k++){
        int u = -1, minDist = INF;
        for (int i = 0; i < G.vexnum; i++){
            if (!include[i] && dist[i] < minDist){              // 寻找当前不在生成树内且距离最小的
邻接点
                minDist = dist[i];
                u = i;
            }
        }
        if (u == -1) break;

        include[u] = true;                                      // 将寻找到的点入队并更新dist
        for (ArcNode *p = G.vertices[u].firstarc; p != NULL; p = p->nextarc) {
            int v = p->adjvex, w = p->weight;
            if (!include[v] && w < dist[v]) {
                dist[v] = w;
                parent[v] = u;
            }
        }
    }
}
```

拓扑排序

```
bool TopoLogicalSort(ALGraph G, int print[]){
    if(G.kind != 1) return false;
    int indegree[G.vexnum] = {0};
    for(int i = 0;i < G.vexnum;i++){
        for(ArcNode *p = G.vertices[i].firstarc;p != NULL;p = p->nextarc) indegree[p->adjvex]++;
    }

    SqStack S;InitStack(S);
    for(int i = 0;i < G.vexnum;i++){
        if(indegree[i] == 0) Push(S, i);
    }

    int count = 0;
    while(!IsEmpty(S)){
        int u;Pop(S, u);
        print[count++] = u;
```

```
        for(ArcNode *p = G.vertices[u].firstarc;p != NULL;p = p->nextarc){
            if(--indegree[p->adjvex] == 0) Push(S, p->adjvex);
        }
    }

    return count == G.vexnum;
}
```

判断图的拓扑序列是否唯一

```
// 有向图的邻接矩阵表示
typedef struct{
    int numVertices, numEdges;
    char VerticesList[MAXV];
    int Edge[MAXV][MAXV];
}MGraph;
int uniquely(MGraph G){
    int count = 0, indegree[G.numVertices] = {0}, v;
    SqQueue Q;InitQueue(Q);
    for(int i = 0;i < G.numVertices;i++){
        int tag = 0;
        for(int j = 0;j < G.numVertices;j++){
            if(G.Edge[j][i] != 0) { tag = 1; indegree[i]++; }
        }
        if(tag == 0) { count++; EnQueue(Q, i); }
    }

    if(count != 1) return 0;                        // 入度为0的结点只能有一个
    while(!IsEmpty(Q)){
        DeQueue(Q, v);
        int tag = 0;
        for(int i = 0;i < G.numVertices;i++){
            if(G.Edge[v][i] != 0){                  // 用邻接矩阵表示需要额外判断v到i是否有边
                indegree[i]--;
                // 根据有向无环图的特性，这里不会访问到已经输出的结点
                if(indegree[i] == 0){
                    if(tag == 1) return 0;
                    tag = 1; EnQueue(Q, i);
                }
            }
        }
    }
    return 1;
}
```

判断两点间是否连通

```
int visited[MaxVertexNum] = {0};
int Pathitoj(ALGraph G, int i, int j){
    if(i == j) return 1;
    visited[i] = 1;
    for(ArcNode *p = G.vertices[i].firstarc;p != NULL;p = p->nextarc){
        int k = p->adjvex;
        if(!visited[k] && Pathitoj(G, k, j)) return 1;
```

```
    }
    return 0;
}
```

查找

折半查找

```
int BinSearch(int A[], int low, int high, int key){
    int mid;
    while(low <= high){
        mid = (low + high) / 2;
        if(key == A[mid]) return mid + 1;
        else if(key < A[mid]) high = mid - 1;
        else low = mid + 1;
    }
    return 0;
}
```

排序

插入排序

直接插入排序

```
// 这里线性表将0位定为哨兵，因此实际长度为length+1
void InsertSort(SqList &L){
    for(int i = 2;i <= L.length;i++){
        L.data[0] = L.data[i];          // 预留的哨兵位
        int j = i - 1;
        while(L.data[j] > L.data[0]){   // 在j=0时一定会跳出
            L.data[j + 1] = L.data[j];
            j--;
        }
        L.data[j + 1] = L.data[0];
    }
}
```

折半插入排序

```
void BinarySort(SqList &L){
    for(int i = 2;i <= L.length;i++){
        if (L.data[i] >= L.data[i - 1]) continue;
        L.data[0] = L.data[i];
        int low = 1, high = i - 1;
        while(low <= high){
            int mid = (low + high) / 2;
            if(L.data[0] < L.data[mid]) high = mid - 1;
            else low = mid + 1;
        }
        // 找到插入位置为low
        for(int j = i;j > low;j--) L.data[j] = L.data[j - 1];
        L.data[low] = L.data[0];
    }
}
```

二路插入排序

```
void TwoRoadSort(SqList &L){
    int n = L.length;
    int temp[MaxSize];                          // 当作容量为n的"循环数组"使用，下标0 ~ n-1
    int first = 0, final = 0;                   // first指向最小值位置，final指向最大值位置
    temp[0] = L.data[1];                        // 原表数据存于L.data[1..n]，先放入第一个元素
    for (int i = 2; i <= n;i++){
        // 待插入元素比最小元素还小：向first前方（逆时针）扩展
        if (L.data[i] < temp[first]){
            first = (first - 1 + n) % n;
            temp[first] = L.data[i];
        }
```

```
        // 待插入元素比最大元素还大：向final后方（顺时针）扩展
        else if (L.data[i] > temp[final]){
            final = (final + 1) % n;
            temp[final] = L.data[i];
        }
        // 待插入元素落在中间：从final一侧向后腾位置，再插入
        else {
            int k = (final + 1) % n;
            while (temp[(k - 1 + n) % n] > L.data[i]) {
                temp[k] = temp[(k - 1 + n) % n];
                k = (k - 1 + n) % n;
            }
            temp[k] = L.data[i];
            final = (final + 1) % n;
        }
    }
    // 从first开始按环序回填到L.data[1..n]，与本章"1起存"约定一致
    for (int k = 0; k < n;k++) {
        L.data[k + 1] = temp[(first + k) % n];
    }
}
```

交换排序

冒泡排序

```
void BubbleSort(SqList &L){
    for(int i = 0;i < L.length - 1;i++){        // 最后一个元素不用遍历
        bool swapped = false;
        for(int j = 0;j < L.length - 1 - i;j++){
            if(L.data[j] > L.data[j + 1]){
                int tmp = L.data[j];
                L.data[j] = L.data[j + 1];
                L.data[j + 1] = tmp;
                swapped = true;
            }
        }
        if(!swapped) break;
    }
}
```

快速排序

```
int Partition(SqList &L, int low, int high){
    L.data[0] = L.data[low];
    while(low < high){
        while(L.data[high] >= L.data[0] && low < high) high--;
        L.data[low] = L.data[high];
        while(L.data[low] <= L.data[0] && low < high) low++;
        L.data[high] = L.data[low];
    }
    L.data[low] = L.data[0];
    return low;
```

```
}
void QuickSort(SqList &L, int low, int high){
    if(low < high){
        int p = Partition(L, low, high);
        QuickSort(L, low, p - 1);
        QuickSort(L, p + 1, high);
    }
}
void Sort(SqList &L){
    QuickSort(L, 1, L.length);
}
```

以所有元素的平均值作为界值

```
void QuickSort(int R[], int low, int high){
    if(low >= high) return;
    int sum = 0;
    for(int i = low;i <= high;i++) sum += R[i];      // 只对当前待划分区间求和
    int avg = sum / (high - low + 1);
    int i = low, j = high;
    while(i <= j){
        // 以阈值做双指针划分，交换不满足阈值的一对元素
        while(i <= j && R[i] <= avg) i++;
        while(i <= j && R[j] >  avg) j--;
        if(i < j){
            int temp = R[i];
            R[i] = R[j];
            R[j] = temp;
            i++; j--;
        }
    }
    if(low < j) QuickSort(R, low, j);
    if(i < high) QuickSort(R, i, high);
}
void Sort(int R[], int n){
    QuickSort(R, 0, n - 1);
}
```

选择排序

简单选择排序

```
void SelectSort(SqList &L){
    for(int i = 1;i < L.length;i++){            // 最后一个元素无需遍历
        L.data[0] = i;
        for(int j = i + 1;j <= L.length;j++){   // 找最小时，最后一个元素依然需要遍历
            if(L.data[j] < L.data[L.data[0]]) L.data[0] = j;
        }
        if(L.data[0] != i){
            int tmp = L.data[i];
            L.data[i] = L.data[L.data[0]];
            L.data[L.data[0]] = tmp;
        }
```

```
    }
}
```

堆排序

```
#define MaxSize 1000
typedef struct{
    ElemType data[MaxSize];
    int size;
}Heap;
void HeapAdjust(int arr[], int k, int len){
    arr[0] = arr[k];
    for(int i = 2 * k;i <= len;i *= 2){
        if(i < len && arr[i + 1] > arr[i]) i++;
        if(arr[0] >= arr[i]) break;
        arr[k] = arr[i];
        k = i;
    }
    arr[k] = arr[0];
}
void HeapSort(int arr[], int len){
    if(arr == NULL || len <= 1) return;
    // 要满足2i为左孩子的特性，下标必须从1开始
    for(int i = len / 2;i > 0;i--) HeapAdjust(arr, i, len);
    while(len > 1){
        int tmp = arr[1];
        arr[1] = arr[len];
        arr[len] = tmp;
        HeapAdjust(arr, 1, --len);
    }
}
```

二路归并排序

```
void Merge(SqList &L, SqList &T, int low, int mid, int high){
    int p = low, q = mid + 1, k = low;
    // 归并到辅助表 T
    while (p <= mid && q <= high) {
        if (L.data[p] <= L.data[q]) T.data[k++] = L.data[p++];      // 取等号保证稳定性
        else T.data[k++] = L.data[q++];
    }
    while (p <= mid) T.data[k++] = L.data[p++];
    while (q <= high) T.data[k++] = L.data[q++];

    // 把归并结果拷贝回原表 L
    for (int i = low; i <= high;i++) L.data[i] = T.data[i];
}

void MergeSort(SqList &L, SqList &T, int low, int high){
    if(low < high){
        int mid = (low + high) / 2;                  // 对半
        MergeSort(L, T, low, mid);
        MergeSort(L, T, mid + 1, high);
        Merge(L, T, low, mid, high);                 // 归并
```

```
    }
}

void Sort(SqList &L) {
    if(L.length <= 1) return;
    SqList T;T.length = L.length;
    MergeSort(L, T, 1, L.length);
}
```

真题汇总

2026

1. 编写代码及思想，有n个人围成一个圈，他们的编号为1~n。现在要求从第s个人开始报数，数到m的人出列，出列后的下一个人继续从1开始报数，同样数到m的人出列，一直重复这个过程，直到圈中剩下最后一个人，返回其编号。（8分）

```
// 思想：循环单链表模拟报数过程；每数到m就摘除当前结点，直到只剩一个
int Josephus(int n, int s, int m){
    // 建立循环单链表，结点编号1~n
    LNode *head = (LNode*)malloc(sizeof(LNode));
    head->data = 1;
    LNode *r = head;
    for(int i = 2;i <= n;i++){
        LNode *p = (LNode*)malloc(sizeof(LNode));
        p->data = i;
        r->next = p;
        r = p;
    }
    r->next = head;                        // 成环
    // 从第s个结点开始报数：先找到s号结点
    LNode *p = head, *pre = r;
    while(p->data != s){ pre = p; p = p->next; }
    // 报数到m出列，直到只剩一个结点
    while(p->next != p){
        for(int i = 1;i < m;i++){ pre = p; p = p->next; }
        pre->next = p->next;               // 摘除p
        free(p);
        p = pre->next;
    }
    return p->data;
}
// 时间复杂度O(nm)，空间复杂度O(n)
```

2. 设一棵二叉树中结点的值互不相同，其先序序列和中序序列分别存于两个一维数组A[1...n]和B[1...n]中，试编写算法建立该二叉树的二叉链表。（10分）

```
// 思想：递归划分——先序第一个元素是根，在中序中定位根的位置，据此把中序分成左右子树，
// 并按左右子树结点数把先序也切分成两段，递归建树
BiTree Build(BiTree &T, int A[], int B[], int preL, int preR, int inL, int inR){
    if(preL > preR) return NULL;
    T = (BiTNode*)malloc(sizeof(BiTNode));
    T->data = A[preL];                     // 先序第一个元素为根
    int k = inL;
    while(B[k] != A[preL]) k++;            // 在中序中定位根的位置
    int leftLen = k - inL;                 // 左子树结点个数
    Build(T->lchild, A, B, preL + 1, preL + leftLen, inL, k - 1);
    Build(T->rchild, A, B, preL + leftLen + 1, preR, k + 1, inR);
    return T;
```

```
}
// 时间复杂度O(n²)（每次定位根需在中序中查找），空间复杂度O(n)（递归栈）
```

3. 编写代码及思想，给一个用邻接表存储的图，给定一个序列V1V2...Vn，判断这个序列是不是该图的一个拓扑序列。（12分）

```
// 思想：入度表法—先求所有顶点的入度；按给定序列顺序，每检查一个顶点u，
// 若当前入度不为0则不是拓扑序列（说明还有前驱未在其前面输出），
// 否则把u的所有后继入度减1，继续检查下一个
bool IsTopoSeq(ALGraph G, int seq[], int n){
    if(G.vexnum == 0) return false;
    int indegree[MaxVertexNum] = {0};
    for(int i = 0;i < G.vexnum;i++)
        for(ArcNode *p = G.vertices[i].firstarc;p != NULL;p = p->nextarc)
            indegree[p->adjvex]++;
    for(int i = 0;i < n;i++){
        int u = seq[i];
        if(indegree[u] != 0) return false;
        for(ArcNode *p = G.vertices[u].firstarc;p != NULL;p = p->nextarc)
            indegree[p->adjvex]--;
    }
    return true;
}
// 时间复杂度O(|V|+|E|)，空间复杂度O(|V|)
```

2025

1. 编写代码及思想，给定一个单链表，找到倒数第k个元素，如果找到该元素，打印该元素的值并返回1，否则返回0。结点定义如下。

```
typedef struct node{
    int data;
    struct node *next;
}LNode, *LinkList;
```

2. 编写代码及思想，给定一棵二叉树，输出从根结点到叶子结点的最长路径，打印路径的结点值，并输出该路径的长度。二叉树的链式存储结构描述如下。

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild, *rchild;
}BiTNode, *BiTree;
void longestPath(BiTree node, int path[], int pathLen, int maxPath[], int &maxLen){
    if(node == NULL) return;
    path[pathLen++] = node->data;
    if(node->lchild == NULL && node->rchild == NULL){
        if(pathLen > maxLen){
            maxLen = pathLen;
            for(int i = 0;i < maxLen;i++) maxPath[i] = path[i];
        }
    }else{
        longestPath(node->lchild, path, pathLen, maxPath, maxLen);
```

```
        longestPath(node->rchild, path, pathLen, maxPath, maxLen);
    }
}
void findLongestPath(BiTree T){
    int path[MaxSize], maxPath[MaxSize];
    int pathLen = 0, maxLen = 0;
    longestPath(T, path, pathLen, maxPath, maxLen);
    for(int i = 0;i < maxLen;i++) printf("%d ", maxPath[i]);
    printf("\n%d", maxLen);
}
```

3. 编写代码及思想，给定一个邻接表表示的连通图，输出从顶点u到顶点v并经过顶点k的一条路径。如果不存在这样的路径输出-1。图的存储结构描述如下。

```
#define MaxVertexNum 100
typedef char InfoType; //附加信息类型
typedef int VertexType; //顶点数据类型

typedef struct ArcNode{//边表结点
    int adjvex;//该弧所指向的顶点位置
    struct ArcNode *next;//下个结点
    //InfoType info;//当前结点（弧）的信息
}ArcNode;

typedef struct VNode{//顶点表结点
    VertexType data;//顶点信息
    ArcNode *firstarc;//指向第一条依附该顶点的弧的指针
}VNode,AdjList[MaxVertexNum];

typedef struct{//图
    AdjList vertices;//邻接表
    int vexnum,arcnum;//图的当前顶点数,图的边数/弧数
}ALGraph;
bool checkPath(ALGraph G, int u, int v, bool visited[]){
    if(u == v) return true;
    visited[u] = true;
    for(ArcNode *p = G.vertices[u].firstarc;p != NULL;p = p->next){
        int k = p->adjvex;
        if(visited[k]) continue;
        if(checkPath(G, k, v, visited)) return true;
    }
    visited[u] = false;
    return false;
}
bool IsExistPathGoBy(ALGraph G, int u, int k, int v){
    if(G.vexnum == 0 || G.arcnum == 0) return false;
    bool visited[G.vexnum] = {false};
    if(!checkPath(G, u, k, visited)) return false;
    for(int i = 0;i < G.vexnum;i++) visited[i] = false;
    return checkPath(G, k, v, visited);
}
```

2024

1. 编写代码及思想，判断一棵树是否为二叉排序树，结构体已经给出。

```
typedef struct BiTNode{
    int data;
    struct BiTNode *lchild, *rchild;
}BiTNode, *BiTree;
int JudgeBST(BiTree T){
    int prev = -1;
    return checkBST(T, prev);
}
int checkBST(BiTNode *node, int &prev){
    if(node == NULL) return 1;
    if(!checkBST(node->lchild, prev)) return 0;
    if(node->data <= prev) return 0;
    prev = node->data;
    return checkBST(node->rchild, prev);
}
```

2. 编写代码及思想，判断在无向图中是否存在长度为k的路径（使用邻接表存储）

```
bool DFS(ALGraph& G, int i, int depth, int k, int visited[]){
    if(depth == k) return true;                 // 当遍历深度等于k时成功
    visited[i] = 1;
    for(ArcNode* e = G.vertices[i].firstarc; e != NULL; e = e->next){
        int v = e->adjvex;
        if(!visited[v]){
            if(DFS(G, v, depth + 1, k, visited)){
                visited[i] = 0;
                return true;
            }
        }
    }
    visited[i] = 0;                             // 该路径寻找失败，回溯清空该结点的访问状态
    return false;
}

bool existsKSimplePath(ALGraph& G, int k){
    if(k < 1 || k > G.vexnum - 1) return false;
    int visited[G.vexnum];
    for(int i = 0; i < G.vexnum;i++){
        visited[i] = 0;
        if(DFS(G, i, 0, k, visited)) return true;
    }
    return false;
}
```

3. 编写代码及思想，计算二叉树的繁荣度（即每一层的结点数乘以该层的层数）

```
// 求树宽度的变体
int prosperity(BiTree T){
    if(T == NULL) return 0;
    int front = 0, rear = -1, last = 0;
```

```
    BiTree Q[MaxSize];BiTree p;
    Q[++rear] = T;
    int totalProsper = 0, width = 0, height = 1;
    while(front <= rear){
        p = Q[front++];width++;
        if(p->lchild != NULL) Q[++rear] = p->lchild;
        if(p->rchild != NULL) Q[++rear] = p->rchild;
        if(front > last){
            // 当一层的循环结束时，累加繁荣度，更新last值和高度值并清空宽度
            totalProsper += height * width;
            last = rear;height++;width = 0;
        }
    }
    return totalProsper;
}
```

2023

1. 已知L为链表的头结点地址，表中共有m(m>3)个结点，从表中第i(1<l<m)个结点起到第m个结点构成一个循环部分链表。试设计一个时间复杂度为O(n)的算法PattemInvertL(LinkList L, int i, int m)，将这部分循环链表中所有结点顺序完全倒置。请给出算法思想和代码。

```
void PattemInvertL(LinkList L, int i, int m){
    if(L == NULL || i <= 1 || i >= m) return;
    LNode *head = L;int j = 0;
    while(j < i - 1){ head = head->next; j++; }
    LNode *p = head->next, *rear = head->next, *temp;
    while(j < m){
        temp = p;
        p = p->next;
        temp->next = head->next;
        head->next = temp;
        j++;
    }
    rear->next = head->next;
}
```

2. 给定一个nm的矩形位图，位图中的每个像素不是白色就是黑色，但至少有一个是白色的。第i行第j列的像素被称作像素(i,j)。两个像素之间的距离定义为 `d(p1,p2)=|i1-i2|+|j1-j2|` 。现在请设计一个时间复杂度尽可能低的算法来计算图中每个像素与其最近的白像素的距离。

- 输入：第1行包含两个整数n和m，用一个空格隔开。接下来n行，每一行都包含一个长度为m的01串。第i+1行第j列的字符若为1，则为白。
- 输出：包含n行，每行有m个用空格隔开的整数。第i行第j列的整数表示(i,j)与其最近的白像素之间的距离。
- 请编写伪代码及思想，并给出该算法的时间复杂度

2022

1. 设计一个算法，将一个用循环链表表示的稀疏多项式分解成两个多项式，使这两个多项式中各自仅含奇次项或偶次项，并要求利用原链表中的结点空间构成这两个链表，要求给出算法伪代码和思想，并说明该算法的时间和空间复杂度。提示，其中稀疏多项式采用的循环链表存储结构定义为：

```
typedef struct PolyNode{
    float coef;     // 单项式的系数
    int exp;        // 单项式的指数
    struct PolyNode *next;
}PolyNode;
```

2. 从根到叶子的最大距离称为树的半径。给定一个无向连通图，写一个算法找出半径最小的生成树，要求给出算法代码和思想，并说明该算法的时间和空间复杂度。

2021

1. 一个先递增后递减的序列称为双调序列，现在请设计一个尽可能高效的算法，找到由n个数组成的一个双调序列中最大的关键值。

```
int binary_search(int A[], int n){
    if(n <= 0) return -1;
    int left = 0, right = n - 1, mid;
    while(left < right){
        mid = (left + right) / 2;
        if(mid >= 1 && A[mid - 1] > A[mid]) right = mid;        // 其左侧元素可能是所求值故不+1
        else{
            if(mid < n - 1 && A[mid + 1] > A[mid]) left = mid + 1;
            else return A[mid];
        }
    }
    return A[left];
}
```

2. 一个正整数序列组成的有序单链表，针对以下三个问题，设计一个时间和空间复杂度最小的算法实现：①确定在序列中比正整数x大的数有几个（相同的数值只计算一次）；②将单链表中比正整数x小的数按递减次序排列；③将比x大的正偶数从单链表中删除。

```
int biggerCount(LinkList L, int x){
    int before = -1, count = 0;
    LNode *p = L->next;
    while(p != NULL){
        int value = p->data;
        if(value > x && value != before) { count++; before = value; }
        p = p->next;
    }
    return count;
}
void reverseSmaller(LinkList &L, int x){
```

```
    if(L == NULL || L->next == NULL || L->next->data >= x) return;
    LNode *p = L;
    while(p->next != NULL && p->next->data < x) p = p->next;
    LNode *q = p->next, *rear = L->next, *temp;
    p->next = NULL;
    p = rear->next;
    while(p != NULL){
        temp = p;
        p = p->next;
        temp->next = L->next;
        L->next = temp;
    }
    rear->next = q;
}
void removeBiggerEven(LinkList &L, int x){
    LNode *p = L, *temp;
    while(p->next != NULL){
        int value = p->next->data;
        if(value > x && value % 2 == 0){
            temp = p->next;
            p->next = temp->next;
            free(temp);
        }else p = p->next;                  // 只有不删除下一个元素的时候才移动p
    }
}
```

2020

1. 将带头结点的双向循环链表L=(a1,a2,a3,...,an)改造为L=(a1,a3,...,an,...,a4,a2)

```
void transformList(LinkList &head){
    if(head == NULL || head->next == head || head->next->next == head) return;
    LNode *rear = head->prior, *p = head->next, *q, *temp;
    while(p != rear && p->next != rear){
        q = p->next;temp = q->next;
        p->next = temp;temp->prior = p;
        q->next = rear->next;q->prior = rear;
        rear->next->prior = q;rear->next = q;
        p = temp;
    }
}
```

2. 一个序列由n(n<=30000)个不大于200的正整数构成，设计一个时间复杂度尽可能小的算法求其逆序对个数

```
int reverseCoupleCount(int A[], int n){
    int temp[n] = {0};
    return MergeSort(A, temp, 0, n - 1);
}
int MergeSort(int A[], int temp[], int left, int right){
    if(left >= right) return 0;
    int mid = (left + right) / 2, count = 0;
    count += MergeSort(A, temp, left, mid);
    count += MergeSort(A, temp, mid + 1, right);
```

```
    count += merge(A, temp, left, mid, right);
    return count;
}
int merge(int A[], int temp[], int left, int mid, int right){
    for(int k = left;k <= right;k++) temp[k] = A[k];        // 先复制到辅助表
    int i = left, j = mid + 1, k = left, count = 0;
    while(i <= mid && j <= right){
        if(temp[i] <= temp[j]) A[k++] = temp[i++];
        else{
            count += mid - i + 1;                          // 左半剩余元素均大于temp[j]，构成逆序
对
            A[k++] = temp[j++];
        }
    }
    while(i <= mid) A[k++] = temp[i++];
    while(j <= right) A[k++] = temp[j++];
    return count;
}
```